Architettura microservizi

1. Introduzione
2. Componenti
   2.1. Microservizi di frontiera
      2.1.1. inbound
      2.1.2. outbound PagoPA
      2.1.3. inbound/outbound
   2.2. Microservizi entità
   2.3. Microservizi processi batch
3. Sicurezza
   3.1. Classi di utenze
4. Multi-Tenancy
5. Storage
   5.1. Condivisione file
   5.2. Database
      5.2.1. Redis
      5.2.2. PostgreSQL
      5.2.3. MongoDB
      5.2.4. Dati personali
         5.2.4.1. Supporto casi d'uso di ricerca
6. Caching
7. Logging e Tracing
8. Test
   8.1. Soak Test
9. Osservabilità
10. Monitoraggio
   10.1. Monitoraggio in Azure

1. Introduzione

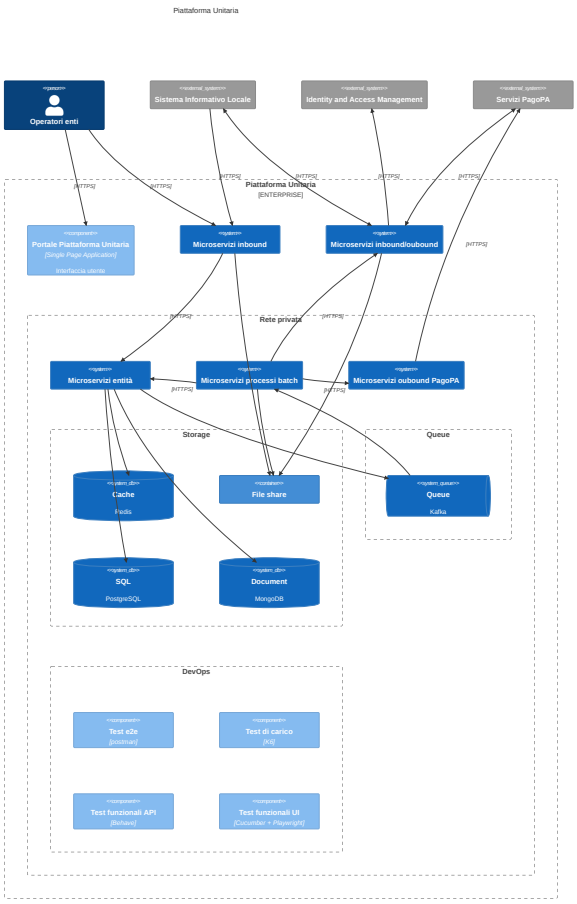
Il presente documento ha lo scopo di descrivere l'architettura a microservizi che verrà installata all'interno dell'istanza di AKS.

Ogni microservizio rappresenterà quindi un container il cui ciclo di vita e relativi log in console sono interamente gestiti da Kubernetes.

Ogni microservizio presenta una delle seguenti classificazioni:

- `TOP`: Microservizi portanti dell'intero sistema, essenziali per l'operatività di base. Rappresentano l'architrave tecnologica senza la quale l'intera infrastruttura collassa.
- `MID`: Microservizi che abilitano il funzionamento efficiente e fluido del sistema. Sono cruciali per le operazioni quotidiane ma non immediatamente letali se temporaneamente non disponibili.
- `EXT`: Microservizi periferici che arricchiscono l'esperienza e forniscono funzionalità aggiuntive. La loro assenza non compromette il funzionamento base del sistema.

2. Componenti



2.1. Microservizi di frontiera

Questa classe di microservizi è dedita a scambiare dati tramite internet.

2.1.1. inbound

Microservizio	Tier	Source	Responsabilità
p4pa-pu-bff	TOP	FE	• Espone i dati verso il FE. • Autentica le sessioni richiedendo un token JWT, validato mediante il servizio **p4pa-auth**. • Autorizza ed applica il cono di visibilità.

2.1.2. outbound PagoPA

Microservizio	Tier	Target	Responsabilità
p4pa-io-notification	EXT	IO	• Gestisce l'invio delle notifiche tramite l'app IO.
p4pa-pdnd-services	MID	PDND ANPR	• Recupera i dati dai servizi integrati tramite PDND. V. Integrazione PDND Interop tramite servizio P4PA-PDND-Services per accedere ai servizi ANPR.

p4pa-send-notification	EXT	SEND	• Invia le notifiche tramite SEND.

2.1.3. inbound/outbound

Microservizio	Tier	Source	Target	Responsabilità
p4pa-auth	TOP	FE SIL	IAM	• Gestisce le sessioni degli utenti del portale FE (V. Autenticazione e Autorizzazione) • Gestisce la registrazione dei client dei SIL e relative sessioni (V. RFC: Autenticazione application-to-application). • Salva utenti e ruoli.
p4pa-pagopa-payments	TOP	NODO-SPC	NODO-SPC	• Scambia i dati con il Nodo dei Pagamenti-SPC. • Autentica le richieste provenienti dal nodo mediante mTLS (nell'installazione in Azure, la terminazione TLS sarà configurata nell'Application Gateway).
p4pa-pu-sil	TOP	SIL	SIL	• Scambia i dati con i SIL. • Autentica le sessioni richiedendo un token JWT, validato mediante il servizio **p4pa-auth**. • Autorizza ed applica il cono di visibilità.
p4pa-fileshare	MID	FE SIL	FE SIL	• Gestisce l'upload ed il download dei file. • Autentica le sessioni richiedendo un token JWT, validato mediante il servizio **p4pa-auth**. • Autorizza ed applica il cono di visibilità.
p4pa-citizen	TOP	ARPU	ARPU	• Scambia i dati con la componente backend di **Area Riservata Piattaforma Unitaria**
p4pa-legal-docs	EXT	ARPU	ARPU	• Gestisce i **Termini di Servizio (ToS)** e **Privacy Policy (PP)** specifici per ciascun broker, con supporto alla localizzazione multilingua.

2.2. Microservizi entità

Questa classe di microservizi è dedita alla gestione dei dati persistiti sui database.

La separazione è stata realizzata rispettando i domini delle entità e la transazionalità delle operazioni.

Microservizio	Tier	Entità	Responsabilità

p4pa-organization	TOP	`broker`
		`organization`
		`org_sil_service`
		`organization_features`
		`taxonomy`
			• Gestisce gli enti:
			  ◦ Carica i loghi;
			  ◦ Censisce le funzionalità abilitate.
			• Gestisce la tassonomia degli enti.
			• Gestisce gli intermediari:
			  ◦ Salva le chiavi usate per la comunicazione con il nodo dei pagamenti (V. RFC: Criptazione api-key Intermediario).

p4pa-debt-positions	TOP	`debt_position_type`
		`debt_position_type_org`
		`debt_position_type_org_operators`
		`debt_position`
		`payment_option`
		`installment`
		`transfer`
		`receipt`
		`iuv_sequence_number`
			• Gestisce le tipologie di Posizioni Debitorie abilitate ai singoli Intermediari.
			• Gestisce l'associazione delle tipologie di Posizioni Debitorie con gli Enti:
			  ◦ Gestisce l'associazione di queste ultime con gli operatori.
			• Gestisce le Posizioni Debitorie:
			  ◦ Creazione, validazione e notifica.
			  ◦ Ricezione ricevuta.

p4pa-classification	MID	`payments_reporting` `treasury`	• Gestisce le Rendicontazioni.

		payment_notification classification assessments assessments_details	• Gestisce la Tesoreria. • Gestisce i Pagamenti Notificati. • Gestisce gli Accertamenti. • Gestisce le Classificazioni.
p4pa-process-executions	MID	ingestion_flow_files export_file	• Gestisce il censimento dei flussi di caricamento massivo.
p4pa-workflow-hub	TOP	workflow_type workflow_type_org debt_position_workflow_type	• Gestisce il censimento e la configurazione dei Workflow Custom (V. Workflow custom per le Posizioni Debitorie).
p4pa-registries	EXT	debt_position_registry installment_registry pagopa_registry sil_registry	• Gestisce il registro delle interazioni con il nodo dei pagamenti. • Gestisce la timeline degli eventi sulle Posizioni Debitorie e sugli Installment

2.3. Microservizi processi batch

Questa classe di microservizi è dedita alla gestione di processi batch e si basano sull'uso di Temporal.io (V. RFC: Gestore workflow (~ Temporal.io)).

Microservizio	Tier	Responsabilità
p4pa-workflow-hub	TOP	• Gestisce la schedulazione dei workflow su temporal.
p4pa-workflow-worker	MID	• Esegue le Activity di temporal.
p4pa-iam-sync	EXT	• Recepisce gli eventi provenienti dall'**IAM** ed allinea enti/operatori

3. Sicurezza

Solamente i microservizi appartenenti alle classi **inbound** e **inbound/outbound** si preoccupano di autenticare, autorizzare e gestire il cono di visibilità delle richieste.

Le API dei restanti applicativi saranno fruibili solamente da rete interna e richiederanno comunque la presenza di un token firmato: a differenza dei precedenti microservizi, questi non gestiranno le autorizzazioni e nel validare i token si limiteranno a verificarne la firma e la data di scadenza in locale, senza invocare il microservizio **p4pa-auth**.

Tutti i microservizi saranno registrati sull'ingress ed in quanto tale saranno fruibili da tutte le risorse collegate alla medesima rete, compresi gli utenti abilitati alla VPN, i quali potranno usufruire delle API usando il medesimo token ottenibile tramite l'accesso al portale.

> Le chiamate interne al cluster di Kubernetes avvengono tramite Service configurati per parlare con protocollo HTTP.
>
> Il token di sessione trasmesso avrà quindi il solo scopo di identificare l'utente dietro la richiesta.

3.1. Classi di utenze

Tutti gli utenti sono autenticati mediante protocollo **OAuth2** con un token di sessione sempre soggetto a scadenza.

Sono associati ad un identificativo tecnico, detto **mappedExternalUserId**, osservabile tramite:

- Introspezione del token di sessione, ossia invocando l'API *getUserInfo* del microservizio **p4pa-auth** (all'interno della cui risposta sarà presente anche il flag `systemUser` valorizzato a `true` per le utenze di sistema).
- Tramite la claim `sub` del payload del token.

> Nei log prodotti è sempre presente il **mappedExternalUserId**, allo scopo di riconoscere sempre l'utente in sessione che ha portato alla produzione del log (V. Logging | 2.1.1. Pattern).

Esistono 3 tipologie di utenti, riconoscibili osservando la struttura del loro **mappedExternalUserId**:

1. Utenti provenienti dall'**IAM esterno** ed autenticati tramite il flusso **TokenExchange** del protocollo **OAuth2** (V. Autenticazione e Autorizzazione).
   - Il loro **mappedExternalUserId** è una stringa Base64.
2. Client associati ai **SIL** e autenticati tramite il flusso **ClientCredentials** del protocollo **OAuth2** (V. Autenticazione application-to-application).
   - Il loro **clientId** segue la forma `<ORG_IPA_CODE><CLIENT_NAME>`
   - Il loro **mappedExternalUserId** segue la forma `WS_USER-<CLIENT_ID>`.
   - Possiede privilegi amministrativi sull'intera organizzazione di appartenenza.
3. **Utenza tecnica di sistema**, autenticata tramite il flusso **ClientCredentials** del protocollo **OAuth2**.
   - I loro **clientId** segue la forma `piattaforma-unitaria_<ORG_IPA_CODE>`.
   - Il loro **mappedExternalUserId** segue la forma `WS_USER-<CLIENT_ID>`.
   - Quando `ORG_IPA_CODE` è popolato, possiede privilegi amministrativi sull'intera organizzazione indicata, altrimenti sarà al di fuori dei coni di visibilità laddove previsti (V. Architettura microservizi | 2.1. Microservizi di frontiera).
   - Questo genere di utenza è in grado di impersonificare gli altri utenti comunicando insieme al token di sistema l'header `X-user-id`, ma mantenendo i permessi amministrativi qualora il token sia stato ottenuto per come descritto al punto precedente
     - Nei log prodotti figureranno sia il **mappedExternalUserId** di sistema che il valore di tale header
       - Il pattern documentato in Logging | 2.1.1. Pattern varierà leggermente in quanto all'interno dei log invece che il singolo campo `[userId]`, comparirà `[systemUserId][userId]`
         - Da notare l'assenza di spazio tra la chiusura delle prime quadre e l'apertura delle successive, la cosa ha impatti nelle query di ispezione documentate in PU - Runbook ispezione Log | 2.3. Query con parsing dei messaggi e PU - Runbook query log per utente/traceId | 2.3. Query per userId e traceId.
     - Se presente, l'header `X-user-id` verrà automaticamente propagato durante le invocazioni REST.

4. Multi-Tenancy

L'entità che individua un tenant rispetto ad un altro è il broker (o intermediario), rappresentante la regione che fornisce le proprie configurazioni nell'accedere ai servizi di PagoPA.

La segregazione è applicata solamente sui dati, utilizzando le stesse tabelle ed usando la colonna `broker_id` per distinguere tra un tenant e l'altro.

5. Storage

5.1. Condivisione file

Esistono casi d'uso che prevedono lo scambio di file con gli attori esterni (utenti del FE e SIL), alcuni file conterranno dati personali.

Questi verranno accettati in chiaro dai relativi microservizi inbound, ma immediatamente cifrati prima di essere depositati all'interno di una share condivisa tra i microservizi (V. RFC: Cifratura file su file system).

Qualunque microservizio si troverà a dovervi accedere, dovrà essere configurato con l'opportuna chiave di decifratura e non dovrà mai esporre il file in chiaro all'interno della share.

5.2. Database

5.2.1. Redis

Utilizzato come cache.

5.2.2. PostgreSQL

Utilizzato per le entità coinvolte in processi che richiedono transazionalità cross-entità.

5.2.3. MongoDB

Utilizzato per le entità per le quali la transazionalità tra i soli record della stessa entità è sufficiente.

Utilizzato per le tabelle di staging alimentate da processi massivi di import dei dati (V. RFC: Database di Staging).

5.2.4. Dati personali

I dati personali non risiederanno contestualmente ai restanti dati, ma verranno estrapolati, cifrati e salvati su un database SQL separato (V. [GAP MyPay Operatore] Persistenza dati sensibili dei cittadini | 2.1.1. Tabelle).

5.2.4.1. Supporto casi d'uso di ricerca

Esistono casi d'uso per i quali si richiede la ricerca di determinate entità in base a campi personali (es. codice fiscale).

Per sopperire a tali requisiti, contestualmente alle entità verrà salvato un hash del dato personale, ammettendo quindi la ricerca esatta per quel dato campo (V. RFC: Algoritmo di Hashing per la ricerca su dati personali).

6. Caching

Al fine di efficientare gli accessi ai dati, sono presenti meccanismi di caching sia locali che distribuiti.

V. Caching per maggiori dettagli.

7. Logging e Tracing

I log sono stampati su console, quindi raccolti e gestiti da AKS stesso.

Sono corredati di un `traceId` generato all'avvio di un processo/richiesta ed inviato durante le richieste REST interne, in modo da poterle relazionare ed individuare con semplicità.

Vedi Logging per maggiori dettagli.

⚠ I log non dovranno mai contenere dati personali

8. Test

La correttezza della soluzione viene continuamente testata all'interno delle pipeline di deploy, sia in fase di merge del codice (Unit Test), sia successivamente al deploy (V. Tipologia di test automatici).

8.1. Soak Test

I livelli di performance sono misurati mediante una suite di soak test (V. PU - Soak tests)

9. Osservabilità

Ciascun microservizio espone degli endpoint REST volti ad indicarne lo stato:

- **liveness**: Indica se il microservizio è correttamente in esecuzione ed in caso di fallimento sarà necessario riavviarlo.
- **readiness**: Indica se il microservizio è in grado di ricevere nuove richieste ed in caso di fallimento questo non potrà accoglierne di nuove.

Kubernetes interrogherà periodicamente queste sonde al fine di gestirne il ciclo di vita.

10. Monitoraggio

Ogni microservizio raccoglie ed espone dati telemetrici mediante la libreria micrometer.

Questi sono esposti tramite console JMX, raggiungibile solamente tramite VPN (si tratta di dati aggregati ed anonimi).

10.1. Monitoraggio in Azure

Ogni microservizio è instrumentato mediante l'agent di Azure ApplicationInsight che si occupa di raccogliere e ed inviare periodicamente i dati telemetrici ad Azure.

Sopra tali dati, insieme ad altri raccolti dalle altre risorse di Azure, sono state poi sviluppate delle dashboard e degli alert volti a mostrare il corretto funzionamento del sistema e la notifica in caso di errore.